

Critter Memory-Focused Unit

Prototype Exemplar and Design-Decision Record

Outlier Removal and Monthly Summarization of Timestamped Temperature Logs

This document records what was built for the memory-focused unit of the Critter product, and — more importantly — **why each choice was made**. It is written to support the workshop-paper deliverable, in which the real product is not the code but the safety-guideline evaluation of the code.

1. Scope: what this unit is, and what it is not

The Critter is a distributed machine-room temperature monitor, functionally decomposed into three units: an I/O unit that acquires readings at a high rate, a **memory-focused unit** that removes outliers and summarizes the data for off-line analysis, and a compute-intensive unit that predicts HVAC behavior. This document covers the middle unit.

Two constraints from the assignment shaped everything below:

- **This is an exemplar, not the product.** Following the guidance that “you are NOT building the application,” the code is a prototypical stand-in for a memory-bound workload, in the same spirit as the matrix-multiply and sorting prototypes used in the undergraduate course. It is the subject of analysis, not the end goal.
- **It must not be optimized.** Per the explicit KISS directive — “write the exemplar without optimization... if you over-engineer and attempt to figure out your own optimization then it will be harder later on” — every opportunity to be clever was deliberately declined. Section 7 lists the optimizations that were consciously left on the table.

The code was also produced by an AI system by design, emulating a junior developer's submission that a team of senior engineers must now review against NASA and SEI CERT guidelines.

2. Deliverables

File	What it is	Role in the assignment
critter_mem.c	The exemplar: 548 lines of C11 (including comments), reads a log, removes outliers, writes a cleaned log and a monthly summary.	The artifact under safety review .
generate_temps.py	Test-data generator. Produces timestamped readings with a known, exact number of planted outliers.	Test harness. Explicitly excluded from the review scope.
sample_temps.txt	1,000,000 readings spanning 2023–2025, in shuffled order, with 40 planted outliers. 26 MB.	Input data and ground truth for verification.

Keeping the generator in a separate language was itself a decision: it makes the review boundary unambiguous. A reviewer cannot accidentally count a finding in the test harness against the unit, and the unit contains no test-only code paths to reason about.

3. Data-model decisions

3.1 Record format

Each reading is one line of plain text, no header:

```
MM/DD/YYYY HH:MM:SS,<temperature_celsius>

09/04/2025 15:02:57,23.846
01/28/2023 16:03:20,19.081
```

Choice: a 24-hour clock with colon-separated time, as specified. **Rationale:** the format is fixed-width and unambiguous, so a single `sscanf` format string parses it, which keeps the parser to one readable line. Plain text was chosen over a binary or packed format because a binary format is an *optimization* — it would shrink the file and speed up loading, which is exactly what the KISS directive rules out.

3.2 In-memory representation

Timestamps are stored as six separate integer fields rather than a packed `time_t` or Unix epoch value:

```
typedef struct {
    int month, day, year;
    int hour, minute, second;
    double temperature;
} Reading; /* 32 bytes */
```

Rationale: converting to epoch seconds would be more compact (8 bytes instead of 24) and would make chronological comparison trivial — both of which are optimizations. Worse, epoch conversion would hide the field the summarization actually needs. Grouping by calendar month requires the month field directly, so storing it directly is the simplest thing that works. The struct lands at 32 bytes with natural alignment.

3.3 Shuffled record order

The sample file is deliberately in **no chronological order**. This was a requested property, and it turns out to be a load-bearing one:

- It proves the summarization is **order-independent**. Monthly bucketing reads the month field of each record, so a shuffled file and a sorted file produce identical summaries.
- It means **no sort is needed** for the summary — which keeps a second `qsort` out of the code. This matters for the safety review, since function pointers are a NASA rule violation (Section 8).
- It is realistic. Readings arriving from several distributed Critter nodes, or merged from multiple logs, would not be globally ordered.

Consequence, stated plainly: the cleaned output file preserves input order, so it is also shuffled. Sorting it chronologically would arguably be friendlier for downstream analysis, but that is a feature addition requiring another sort, and it was left out on KISS grounds. This is a reasonable point to raise in the paper as a deferred design decision rather than an oversight.

4. Dataset sizing

The assignment's guidance was that the data volume is not the point — the unit is “capturing temperature data to evaluate trends and predict HVAC issues.” The size therefore needed only to be large enough that the unit is genuinely *memory*-bound rather than compute-bound. Measured behavior:

Readings	Input file	Peak RSS	Runtime	Bytes / reading
100,000	2.6 MB	9.6 MB	0.32 s	101
1,000,000	25.7 MB	78.4 MB	2.92 s	82
4,000,000	103.0 MB	307.3 MB	14.39 s	81

Footprint grows linearly at roughly **80 bytes per reading**, against a theoretical 72 bytes: two 32-byte record buffers (raw and kept) plus an 8-byte-per-reading sorted copy of the temperatures for the median. The remainder is allocator and program overhead. Extrapolating:

- 10,000,000 readings → roughly 800 MB working set
- 50,000,000 readings → roughly 4 GB working set

Choice: 1,000,000 readings for the shipped sample, spanning three years (2023–2025) at an average of one reading every 95 seconds. **Rationale:** it is a 26 MB file — small enough to hand around and version, large enough that the process holds ~78 MB and the linear scaling is unmistakable. The generator accepts a count argument and the unit imposes no internal ceiling, so the data set can be scaled to whatever the target platform will bear (Section 5):

```
python3 generate_temps.py 10000000 40 temps_big.txt
./critter_mem temps_big.txt
```

That ceiling-free design is itself worth noting in the review: the buffers are sized from whatever the input file happens to contain, with no compile-time maximum. Benign here; a NASA Rule 2 problem elsewhere (Section 8).

5. Target platform: Raspberry Pi 5, 8 GB

The deployment target is a Raspberry Pi 5 with 8 GB of LPDDR4X. Three properties of that board change how the exemplar should be read, and one of them materially changes a safety finding.

5.1 Memory budget

The BCM2712 pairs a quad-core Cortex-A76 at 2.4 GHz with 512 KB of L2 per core and a 2 MB shared L3, behind a 32-bit LPDDR4X interface rated at up to 17 GB/s. Of the 8 GB, roughly 1 GB should be reserved for Raspberry Pi OS with a desktop, and far less headless, leaving on the order of 6.8 GB for this process. At the measured 80 bytes per reading:

Readings	Input file	Working set	Verdict on an 8 GB Pi 5
1,000,000 (shipped)	26 MB	78 MB	Trivial
10,000,000	260 MB	~800 MB	Comfortable
25,000,000	650 MB	~2.0 GB	Comfortable
50,000,000	1.3 GB	~4.0 GB	Fits, roughly 3 GB spare
85,000,000	2.2 GB	~6.8 GB	At the edge — OOM risk

The input and cleaned-output files also occupy page cache, but the kernel evicts that under pressure, so it does not count against the process. **Practical ceiling: roughly 80 million readings**, with 50 million the largest round number that leaves real headroom.

5.2 Where the time actually goes

It is tempting to assume a memory-focused unit is limited by memory bandwidth. Instrumenting the phases shows otherwise. For the one-million-reading sample:

Phase	Time	What dominates it
Pass 1: count lines	0.026 s	File I/O only
Pass 2: read and parse	0.820 s	sscanf text-to-binary conversion
Mean and standard deviation	0.006 s	Two streaming passes over 32 MB
Filter, stats, median	0.418 s	Mostly qsort
Write cleaned file	0.669 s	fprintf binary-to-text conversion
Summarize by month	0.004 s	One pass, twelve accumulators

This is arguably the most useful measurement in the document. The statistics passes — the part that looks expensive — cost six milliseconds. Roughly three quarters of the runtime is ASCII-to-binary and binary-to-ASCII conversion. The unit is therefore **memory-bound in footprint but parse-bound in time**: its RAM scales linearly and unavoidably with the data, while its wall clock is set by text formatting. That reorders the optimization priorities — a binary file format would buy far more than anything done to the statistics loops — which is useful to know and, per the KISS directive, deliberately not acted on.

A related inefficiency worth naming: the record is 32 bytes, but the statistics loops read only the 8-byte temperature field, so three quarters of every 64-byte cache line fetched is wasted. Splitting timestamps and temperatures into parallel arrays would fix it. At this scale it is immaterial, because those loops are already nearly free; it would begin to matter only once the parsing cost was removed.

These figures were measured on x86-64, not on the Pi. Absolute times on a Cortex-A76 will be longer; the *proportions* hold, because they follow from the algorithm rather than the processor. Per-reading memory is identical on both, since `sizeof(Reading)` is 32 bytes under either ABI.

5.3 A 64-bit target makes one finding benign, and portability makes it dangerous

The Cortex-A76 cores are 64-bit only, and Raspberry Pi OS ships 64-bit by default on the Pi 5, so `size_t` is 64 bits. The unchecked multiplication `line_count * sizeof(Reading)` would need roughly 5.8×10^{17} readings to overflow, which is unreachable. **On this platform that CERT INT30/INT32 finding is purely theoretical.**

Port the same source to a 32-bit ARM target — a Pi Zero, an older Pi running a 32-bit userland, or any of the many 32-bit embedded Linux boards — and `size_t` becomes 32 bits. The product then wraps at **134,217,728 readings**. The consequence is not a clean allocation failure: `malloc` receives a small wrapped size, succeeds, and the read loop walks off the end of the buffer. A capacity limit becomes silent heap corruption.

This is a concrete instance of the broader question the assignment raises. The *same line of code* is an untriggerable edge case on the Critter's actual hardware and a memory-safety defect one port away. A severity rating assigned against the current target does not survive a change of target, which argues for rating findings against the *portability envelope* rather than against the board on the bench.

5.4 The malloc NULL check is weaker than it looks

The code checks both allocations for NULL and exits cleanly on failure. Under Linux's default heuristic overcommit, that check is only partial protection. Measured behavior:

```
RSS before malloc : 1 MB
malloc(1 GB) succeeded : the NULL check PASSES
RSS after malloc : 1 MB <-- nothing committed yet
RSS after first touch : 1025 MB <-- committed only now
```

malloc reserves address space; physical pages are committed on first touch. The NULL check confirms the *reservation*, not the *availability*. A grossly impossible request (64 GB on an 8 GB board) is refused outright, so the check is not useless — but a plausible-yet-unavailable request succeeds, memory then runs out during `read_readings`, and the failure arrives as a SIGKILL from the OOM killer. At that point the program cannot report an error, flush a partial result, or run the careful cleanup path written for exactly this case.

Why this matters for the evaluation. A reviewer ticking a checklist sees allocation failure handled and moves on. The finding is not that the check is missing; it is that the check *cannot* deliver what it appears to. This is the strongest argument available for NASA Rule 3's ban on heap allocation after initialization: on an overcommitting OS, a large runtime allocation has no reliable failure mode. It also shows why guideline compliance resists automation — no analyzer flags a correct NULL check, and seeing why it is insufficient requires knowing how the operating system commits memory.

6. Algorithm decisions

6.1 Outlier detection: 3-sigma z-score

A reading is discarded when it lies more than three standard deviations from the mean of the full raw data set.

Why this method: it is the textbook approach, it is two lines of arithmetic, and it requires no tuning constants beyond the threshold itself. Alternatives were rejected as over-engineering: a modified z-score using median absolute deviation is more robust to heavy contamination; an interquartile-range filter needs a sort; a seasonal or rolling-window filter would catch *local* anomalies the global filter misses. Each is a better algorithm and each is an optimization.

Known limitation, deliberately retained: the threshold is computed from statistics that the outliers themselves contaminate. Forty gross faults in a million readings inflate the standard deviation by well under 1%, so it does not matter here — but at higher contamination the filter would start to degrade. This is a genuine finding for the paper, not a bug to patch.

6.2 Verification against ground truth

Because the generator plants an exact, known number of outliers, the filter's correctness is checkable rather than assumed. The measured result on the shipped sample:

Check	Result
Outliers planted by the generator	40
Outliers found and removed by the unit	40
Removed set identical to planted set	Yes

Check	Result
False positives (real readings discarded)	0
Missed outliers	0
Input / cleaned line counts	1,000,000 / 999,960
Temperature range surviving the filter	15.010 – 28.992 °C

The planted faults sit at 78.9 to 97.3 °C and –45.0 to –28.5 °C, while the synthetic legitimate band is about 15 to 29 °C and the computed 3-sigma window is [12.275, 31.714] °C. The separation is wide, which is why the match is exact — the test is designed to be unambiguous, not to be hard.

6.3 Statistics: separate passes

Mean, standard deviation, and median are each computed in their own loop over the data. A single-pass Welford algorithm would compute mean and variance together, touch memory once instead of twice, and be more numerically stable. It was **not** used, precisely because it is the optimization a reader would expect to find later.

6.4 Summarization: by calendar month, all years pooled

Surviving readings are grouped into twelve buckets — every January together, every February together, and so on across the whole log. Each bucket reports count, mean, minimum, and maximum.

Rationale: this is the aggregation that exposes the seasonal signal an HVAC analysis cares about, and it compresses a million readings into twelve rows, which is the whole point of a data-reduction unit. Pooling across years also means more samples per bucket and a smoother trend than a per-month-per-year breakdown would give. The measured output on the shipped sample shows the expected curve — a December trough near 18.1 °C rising to a June peak near 25.9 °C:

Month	Count	Mean °C	Min °C	Max °C
January	85,191	18.444	15.080	21.929
February	77,420	19.740	16.039	23.510
March	84,996	21.604	17.615	25.659
April	82,164	23.628	19.705	27.493
May	84,411	25.230	21.546	28.703
June	82,260	25.937	22.762	28.992
July	85,014	25.589	22.118	28.922
August	84,685	24.248	20.330	28.024
September	82,207	22.315	18.364	26.288
October	85,029	20.298	16.400	24.257
November	82,093	18.732	15.253	22.351
December	84,490	18.059	15.010	21.201

February's lower count is not a defect: it has fewer days, so uniformly random timestamps land in it less often. Worth mentioning because it is the kind of result a reviewer might flag as a bucketing bug on first read.

7. Optimizations deliberately declined

This section exists so a reader can tell **intentional simplicity** from oversight. Each item below is a known improvement that was consciously not made, in direct service of the KISS directive.

What the code does	The optimization not taken	Why it was declined
Loads the entire data set into RAM at once	Stream the file in fixed-size chunks; never hold it all	Streaming is the central memory optimization for this workload. Building it in now would pre-empt later coursework.
Allocates separate raw and kept buffers	Filter in place, compacting survivors into the same buffer	Halves peak memory. Also makes the data flow harder to follow.
Copies temperatures into a third buffer to find the median	Use quickselect, $O(n)$ and no full sort; or reuse an existing buffer	A sorted copy is obviously correct and obviously wasteful — a good thing for a reviewer to see.
Reads the file twice (count, rewind, load)	A single pass into a geometrically growing buffer	Two passes make the allocation size exact and the logic linear. One pass needs realloc bookkeeping.
Separate loops for mean and standard deviation	Welford's single-pass algorithm	Fewer memory passes and better numerical stability, but it obscures the arithmetic.
Stores six int timestamp fields per record	Pack into one time_t, or a 32-bit epoch offset	Cuts the record from 32 to 16 bytes. Also hides the month field the summary needs.
Plain-text input and output	A packed binary format	Far smaller and faster to parse, but unreadable and unreviewable by eye.

8. Implications for the safety evaluation

The reason this unit is interesting is that moving from a bare array of doubles to timestamped records **widened the guideline surface**. Parsing text into a struct introduces string-handling and input-validation concerns that the earlier numeric-only version did not have, and writing two output files rather than one adds file-I/O obligations. The table below maps what is present in the current code.

Observation in the code	CERT C	NASA P10	Found how
Heap allocation of both record buffers, after startup	MEM	Rule 3 — violated	Manual
line_count * sizeof(Reading) can overflow size_t on a huge file	INT30, INT32	—	Automated
Month field from sscanf is used as an array index with no range check	ARR30, INT	Rule 7	Manual
Other timestamp fields unvalidated (day, hour, minute, second)	STR, INT	Rule 7	Manual
Fixed 128-byte line buffer; longer lines silently split	STR, FIO	—	Manual

Observation in the code	CERT C	NASA P10	Found how
Division by <code>std_dev</code> with no zero-check (constant input $\Rightarrow$ divide by zero)	FLP03	Rule 7	Manual
Division by count in mean and summary	FLP03	Rule 7	Manual
<code>qsort</code> requires a comparison function pointer	—	Rule 9 — violated	Manual
Zero <code>assert()</code> calls in the entire file	—	Rule 5 — violated	Automated (count)
Loop bounds derive from runtime file size, not a compile-time maximum	—	Rule 2 — arguable	Manual
<code>compute_median</code> returns 0.0 on allocation failure, hiding the error	ERR, MEM	Rule 7	Manual
<code>fclose</code> and <code>fprintf</code> return values unchecked on both output files	FIO	Rule 7	Automated
Malformed input lines are skipped silently, with only a count reported	ERR	Rule 7	Manual

The headline asymmetry. Of the thirteen observations above, three are reliably caught by tooling and ten require a human reading the code against a rule list. Nearly every *CERT* memory and integer finding is machine-detectable, while nearly every *NASA Power of Ten* finding is structural and manual — no analyzer will tell a team that heap allocation after initialization is forbidden, because that is a policy about architecture, not a detectable defect. That gap is the substantive result to carry into the paper's difficulty metric.

Note also that the code **compiles clean under** `-Wall -Wextra -Wpedantic -std=c11` **with zero warnings** and produces correct results on valid input. Every finding above sits underneath a clean build. That, on its own, is an argument about how far compiler diagnostics go toward safety assurance.

9. The Critter is not safety-critical — but its siblings might be

The same exemplar yields two very different verdicts depending on the product it lands in, and the reason is structural: **the idiomatic way to write data-reduction code in C leans on exactly the features that safety-critical standards forbid**. Large heap buffers, a library sort reached through a function pointer, and loop counts driven by runtime input are all natural, all fine for a benign monitor, and all disallowed under NASA-style rules.

- **For the Critter** (not safety-critical): *CERT* is the appropriate bar. Bounded, checked dynamic allocation is acceptable, tooling does most of the work, and hardening is mostly local edits — range-check the month field, guard the divisions, check the I/O returns. Difficulty: **moderate and tool-assisted**.
- **For a safety-critical sibling** — avionics under DO-178C, automotive under ISO 26262, medical under IEC 62304 — the *Power of Ten* rules apply and this unit must be **redesigned, not patched**: statically pre-allocated buffers sized to a compile-time maximum, a bounded or streaming summarization pass, no `qsort`, no recursion, and assertions throughout. Difficulty: **high, and concentrated in exactly this unit** rather than in the compute unit, whose arithmetic maps onto the rules far more cleanly.

The design takeaway. Even though the Critter does not require it, writing this unit so that it *would also* satisfy the stricter rules — bounded static buffers, no allocation after initialization — costs very little at prototype stage and makes the code portable to a safety-critical variant later. Consistent with the KISS

directive, that was not built in here; but surfacing the trade-off is the design-stage reasoning the assignment is asking for.

10. Reproducing these results

```
# build (zero warnings expected)
gcc -O2 -Wall -Wextra -Wpedantic -std=c11 -o critter_mem critter_mem.c -lm

# run against the shipped sample
./critter_mem sample_temps.txt

# regenerate the data, or scale it up (fixed seed, reproducible)
python3 generate_temps.py 1000000 40 sample_temps.txt
python3 generate_temps.py 10000000 40 temps_big.txt
```

Outputs, written to the working directory:

- `critter_cleaned.txt` — surviving readings, same format as the input, input order preserved
- `critter_summary.csv` — twelve rows of month, month name, count, mean, minimum, maximum

The console output reports the before and after statistics, an itemized list of every discarded reading with its z-score, and the total removed — which should read 40 against the shipped sample.

Compiled and verified on GCC with `-std=c11`; target platform Raspberry Pi 5 (8 GB). All figures in Sections 4, 5 and 6 are measured, not estimated.